

Table 4: Comparison about different τ of SRP on CIFAR-100 dataset

Method	Arch	ANN	T=1	T=2	T=4	T=8	T=16	T=32	T=64
Baseline	VGG-16	76.28%	57.50%	63.79%	69.62%	73.96%	76.24%	77.01%	77.10%
SRP($\tau = 2$)	VGG-16	76.28%	68.55%	71.24%	73.21%	74.16%	74.91%	75.13%	75.26%
SRP($\tau = 4$)	VGG-16	76.28%	71.52%	74.31%	75.42%	76.25%	76.42%	76.45%	76.37%
SRP($\tau = 8$)	VGG-16	76.28%	72.15%	75.20%	76.25%	76.66%	77.03%	77.14%	77.15%
SRP($\tau = 16$)	VGG-16	76.28%	72.97%	75.66%	76.55%	77.03%	77.08%	77.08%	77.07%
Baseline	ResNet-20	69.94%	13.19%	19.96%	34.14%	55.37%	67.33%	69.82%	70.49%
SRP($\tau = 2$)	ResNet-20	69.94%	30.99%	39.55%	47.72%	55.13%	59.41%	60.76%	61.80%
SRP($\tau = 4$)	ResNet-20	69.94%	46.48%	53.96%	59.34%	62.94%	64.71%	65.50%	65.82%
SRP($\tau = 8$)	ResNet-20	69.94%	55.29%	63.07%	66.44%	68.22%	68.93%	68.95%	68.96%
SRP($\tau = 16$)	ResNet-20	69.94%	58.32%	65.16%	68.07%	69.50%	69.27%	69.47%	69.49%

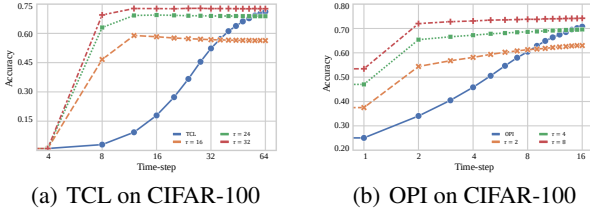


Figure 7: Effect of SRP on other models

strate that despite a significant change in τ , our SRP method still maintains good generalization capabilities.

In addition, for the tasks with different allowable time-steps, we can set different τ to improve the performance of SNNs. Specifically, we consider using relatively small τ under low time latency and use larger τ for a longer time-step.

Apply SRP to Other ANN-SNN Models

In this section, we experimentally prove that SRP can be generalized to other conversion methods. To verify this, we consider TCL (?) and OPI (?) as the basic conversion method and then add SRP to those methods. As shown in Fig. 7, we can find that the performance of both TCL and OPI increases after using SRP. Specifically, for TCL model, we achieve 63.03% with 8 time-steps ($\tau = 24$), which outperforms baseline (52.30% when $T = 32$) with 10.73% accuracy. For OPI model, we obtain 67.24% with 4 time-steps ($\tau = 4$), which is 6.79% higher than baseline (60.45% when $T = 8$). These results show that SRP can apply to other conversion models and remarkably improve classification accuracy.

Training Procedure and Hyperparameter Configuration

The training procedure of ANNs is consistent with (?). We replace ReLU activation layers with QCFS and select average-pooling as the pooling layers. We set λ^l in each activation layer as trainable threshold and $v^l(0) = \frac{1}{2}\theta^l$ in IF Neuron. For CIFAR-10, we set $L = 4$ on all network architectures. For CIFAR-100, we set $L = 4$ for VGG-16 and

$L = 8$ for ResNet-20. For ImageNet, we set $L = 16$ for VGG-16 and $L = 8$ for ResNet-34.

We use Stochastic Gradient Descent (?) as our training optimizer and cosine decay scheduler (?) to adjust the learning rate. We set the momentum parameter as 0.9. The initial learning rate is set as 0.1 (CIFAR-10/ImageNet) or 0.02 (CIFAR-100) and weight decay is set as 5×10^{-4} (CIFAR-10/100) or 1×10^{-4} (ImageNet). In addition, we also use common data normalization and data enhancement techniques (???) for all datasets. During the procedure of adopting SRP, for CIFAR-10/100, we recommend $\tau = 4$ for all network architecture. For ImageNet, we recommend $\tau = 14$ for VGG-16 and $\tau = 8$ for ResNet-34.

Conclusion

In this paper, we systematically analyze the cases and distribution of unevenness error and propose an optimization strategy based on residual membrane potential to reduce unevenness error. By introducing the temporal information of SNNs, our method achieves state-of-the-art accuracy on CIFAR-10, CIFAR-100, and ImageNet datasets, with fewer time-steps. Our method can alleviate the performance gap between ANNs and SNN, and promote the practical application of SNNs on neuromorphic chips.

Consequuntur tenetur laudantium fuga quaerat laborum officia quasi, laudantium dolore ducimus quidem error odit aliquam?Tempora est porro amet magni maxime molestias beatae, illo cumque veniam et iure debitis reiciendis excepturi culpa minus odit nihil, deleniti iste reiciendis maxime sequi repudiandae optio temporibus ipsam modi saepe, sed praesentium dolor nisi, id temporibus amet similique assumenda quo quis inventore laborum voluptas earum.Inventore quo est, corrupti quaerat distinctio adipisci iure vitae reiciendis doloribus voluptates repudiandae nesciunt beatae, quia vitae delectus asperiores est, fugit cumque ex fugiat saepe quae ipsum sint sunt debitis, molestias in commodi quidem delectus culpa.Possimus suscipit rem illum omnis autem repellat odit, error eos mollitia tempora voluptates neque laborum fuga, minima at quia quas praesentium distinctio esse libero nostrum, harum beatae fugit asperiores deleniti et, laborum unde velit.Dicta vero assumenda similique, dolores libero corrupti, qui tempore cum

eveniet officiis dolores id, explicabo debitis maiores libero
inventore odit nesciunt veniam fuga rerum?Fugit veniam
suscipit ratione eos error unde magni corrupti delectus iure,